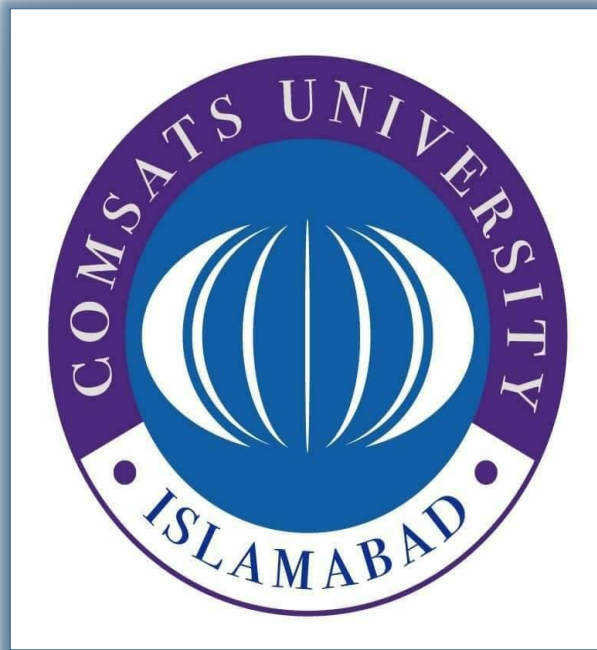




COMSATS University Islamabad

Lahore Campus — Department of Computer Science

CSC241 — Object Oriented Programming (Java)

Theory Assignment 1

Object Oriented Analysis of a Smart Campus Allocation System

Student name	Maryam Mumtaz
Registration number	SP26-BAI-031
Section	BAI (A)
Course	CSC241 — Object Oriented Programming (Java)
Instructor	Dr Muhammad Shahid Bhatti
Assessment	Theory Assignment 1 (Mapped CLO: CLO1)
Date submitted	26 September 2026
Student specific condition	Privacy first (last digit of registration number = 0)

Task 1 — Stakeholder and Requirement Analysis

1.1 Stakeholders and their primary needs

Stakeholder	Primary need
Student	Fair, predictable access to classrooms, labs, seminar rooms and project spaces for individual and group academic work, without having to reveal personal details to other students.
Faculty member	Priority for scheduled teaching sessions and the ability to make urgent academic bookings that ordinary student requests cannot block.
Administrator	High utilization of learning spaces and reliable reporting, to plan capacity and answer to the university for how space is used.
Maintenance staff	The authority to immediately block a room that becomes unsafe — even one with confirmed reservations — without waiting for administrative sign-off.
Security staff	A traceable, timestamped record of who accessed which space and when, to investigate incidents and control building entry.

1.2 Genuine requirement conflicts

Conflict 1 — Fair access (students) versus priority (faculty). Students expect equal treatment, but faculty need guaranteed priority for scheduled teaching and urgent academic activity.

Decision needed: *a precise, published ordering that defines what “fair” means once two different stakeholder classes compete for the same slot.*

Assumption adopted: requests are grouped into five priority tiers — (1) scheduled teaching, (2) urgent faculty academic activity, (3) approved class/academic events, (4) group meetings, (5) individual activity. “Fair access” is defined as equal treatment of requests within the same tier.

Conflict 2 — High utilization (administration) versus immediate safety blocking (maintenance). Administration wants rooms booked as often as possible; maintenance must be able to pull a room out of use the instant it is unsafe, even mid-semester.

Decision needed: *whether a safety block can ever be overridden, and by whom.*

Assumption adopted: a maintenance block takes effect instantly and automatically and cannot be overridden by any role, including the administrator.

Conflict 3 — Traceable access records (security) versus personal privacy (ordinary users). Security needs to know who used a space and why; ordinary users expect their identity and purpose to stay private from other users.

Decision needed: *exactly what reservation detail is visible to whom.*

Assumption adopted (student specific condition — Privacy first): an ordinary user browsing availability sees only whether a space and time slot is free or taken — never the identity of the requester or the purpose of someone else's reservation.

1.3 Facts versus assumptions

The scenario gives the following as facts, stated directly by the university:

- The system covers classrooms, laboratories, seminar rooms and project spaces.

- Four request types exist — individual, group meeting, class, and academic event — each with different capacity, equipment and approval rules.
- A person may hold more than one role at once, and roles and permissions may change during the semester.
- Emergency events may cancel or relocate confirmed reservations; the system must keep an audit trail and notify affected users.
- Maintenance staff must be able to block unsafe rooms immediately; security needs traceable access records; users expect personal information to stay private.

The following are assumptions this submission introduces, because the university left them undecided:

- The five-tier priority ordering used to resolve Conflict 1.
- Maintenance blocks are absolute and cannot be overridden by any role (Conflict 2).
- The identity/purpose visibility rule for ordinary users versus security/administrators (Conflict 3, the Privacy-first condition).
- A concrete recurring-booking limit: at most three standing weekly bookings per user per space type, to give meaning to “limits on recurring reservations.”
- Simultaneous requests for the same slot are resolved first by priority tier, then by the timestamp of the validated submission within that tier.

Task 2 — Object Oriented Conceptual Model

2.1 Proposed classes

Ten classes are proposed (two of them, Classroom and Laboratory, are concrete subclasses of the abstract LearningSpace, used to justify inheritance in the diagram below).

Class	Responsibility	Key attributes / operations
User	Represents any person using the system and holds their identity and session.	userId; name (hidden); contactInfo (hidden) — authenticate(); viewAvailability(); requestReservation()
Role	Represents one assignment of a role type (student, faculty, admin, security, maintenance, disability support) to a User, with its own validity window and permissions.	roleType; validFrom/validUntil; permissionSet — hasPermission(action); isActive()
LearningSpace (abstract)	Represents a physical space and its own availability/safety status, independent of who books it.	spaceId; capacity; status — checkAvailability(slot); block(reason)
Classroom (extends LearningSpace)	A teaching space with classroom-specific rules and features.	boardType; isAccessible — approveForClassSession()
Laboratory (extends LearningSpace)	A specialised space with safety and equipment rules a plain classroom does not have.	labType; safetyRequirements — approveForLabSession()
Reservation	Represents one claim on one space for one time slot on behalf of one	reservationId; purpose (hidden); timeSlot; status — confirm();

	requester, and owns its own life cycle.	cancel(); relocate(newSpace)
ApprovalPolicy	Owns the business rule for evaluating a request and resolving conflicts between competing requests — kept out of Reservation and User.	activityType; priorityRules — evaluate(request); resolveConflict(a, b)
Equipment	Represents a piece of movable or fixed equipment that can be allocated to a reservation, separately from the room itself.	equipmentId; type; isMovable — reserveWith(reservation); checkAvailable()
NotificationService	Owns all outgoing communication to users, so no other class needs to know how messages are sent.	channelType; templates — notify(user, message); broadcastToAffected(event)
AuditLog	Owns the permanent, append-only history of actions, for security traceability and emergency audits.	logId; entries — record(action); generateAuditTrail()

2.2 UML class diagram

The diagram shows associations with multiplicities, an inheritance pair (Classroom and Laboratory specialise the abstract LearningSpace), a composition (a Role assignment has no meaning without the User who holds it, so its lifecycle is owned by User), and an aggregation (Equipment exists and can be reused independently of any one Reservation).

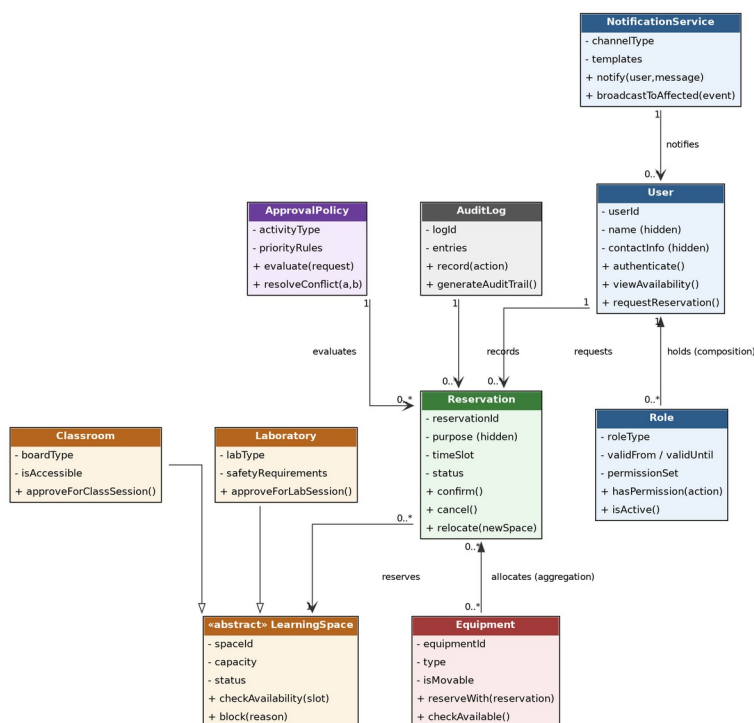


Figure 1 — UML class diagram of the Smart Campus Learning Space Allocation System

2.3 How responsibility is separated

No single class controls the whole system. Each class owns exactly one concern, following the single-responsibility idea:

- User only manages identity, authentication and the roles it holds — it never decides whether a booking is allowed.
- ApprovalPolicy owns the business rule for evaluating requests and resolving conflicts, so that rule can change without touching Reservation, User, or LearningSpace.
- LearningSpace is the sole authority over its own safety/availability status — even a confirmed Reservation cannot force a blocked space to reopen.
- Reservation owns only booking state and its own life cycle transitions (confirm, cancel, relocate).
- NotificationService owns communication, so no other class needs to know how or where a message is delivered.
- AuditLog owns the permanent history, kept separate so that cancelling or relocating a Reservation can never quietly erase what happened.

Because these classes collaborate through method calls on well-defined associations rather than reaching into each other's private data, a change to one responsibility (for example, changing the priority rule) touches one class instead of rippling through the whole model.

Task 3 — Abstraction, Encapsulation and Invariants

3.1 What each selected class abstracts

User — abstracts a real person interacting with the system, regardless of how many roles they currently hold or how those roles change over the semester.

Reservation — abstracts a specific claim on a specific space for a specific time window, on behalf of a specific requester, independent of how that claim is displayed to other users.

LearningSpace — abstracts the physical resource itself and its current safety/availability status, independent of any particular booking.

3.2 Hidden information and controlled change

Class	Hidden information	Only changed through
User	name and contactInfo (raw personal details); the detailed permission set of each Role.	authenticate() to confirm identity; role changes only through administrator-only operations such as assignRole()/revokeRole(), never by editing fields directly.
Reservation	requestedBy (identity) and purpose, hidden from ordinary users under the Privacy-first condition; only status and timeSlot are exposed publicly.	confirm(), cancel(), relocate(newSpace) — every transition is a guarded method that checks the current state first; status can never be set directly.
LearningSpace	the internal status field and any maintenance notes.	block(reason), restricted to the maintenance role, and internal updates driven by confirmed Reservation objects — never edited directly by a plain User.

3.3 Invariants that must always hold

- Every Reservation references exactly one LearningSpace and exactly one requesting User; a Reservation cannot exist without both (the constructor requires both references, matching the 1 multiplicities in Figure 1).

- A LearningSpace can never hold two Confirmed reservations whose time slots overlap — checked inside confirm() before the state is allowed to change, so no double-booking can occur.
- A Role's validUntil is never earlier than its validFrom, and a Role outside its valid window grants no permission — enforced inside isActive(), which hasPermission() always checks first.
- Once a Reservation leaves the Requested/UnderReview states, its identity and purpose fields become read-only to everyone except Security and Administrator roles — enforced by accessor methods that check the caller's role before returning the private fields.
- A cancelled, rejected, or relocated Reservation is never deleted; only its status changes, and AuditLog retains the full history — enforced by never exposing a delete() operation, only state-changing methods.

Encapsulation is what makes these invariants reliable: because the fields involved are private, the only way to change them is through a method that checks the current state and the caller's permission first. That means an invalid transition — confirming an already-cancelled reservation, or a student reading another student's reservation purpose — is rejected at the one place the rule lives.

Task 4 — Alternative Designs and Decision

4.1 Two ways to represent users with multiple, changing roles

Option A — Inheritance. A base class Person with subclasses Student, Faculty, Administrator, SecurityStaff, MaintenanceStaff and DisabilitySupportStaff, each hard-coding its own permissions and behaviour.

Option B — Composition / role objects (the design used in Task 2). A single User class holds a collection of Role instances; each Role carries its own type, validity window and permission set, and can be added or removed independently of the others.

4.2 Comparison

Criterion	Option A — Inheritance	Option B — Composition / role objects
Extensibility	Poor — a new role combination (e.g. a student who is also a teaching assistant) needs a brand-new subclass, or multiple inheritance, which Java does not support for classes.	Good — a new role is simply a new Role instance/type; User itself never changes.
Coupling	High — behaviour is compiled into the class hierarchy, so changing one role's rule can force changes in subclasses or a shared ancestor.	Low — User depends only on the generic Role interface, not on any specific concrete role.
Duplicate behaviour	Likely — shared logic between related roles (e.g. Faculty and a Faculty-as-Coordinator variant) must be copied or awkwardly pulled into a common ancestor.	Minimal — shared logic lives once, inside Role or a shared policy object that every Role instance reuses.
Multiple simultaneous roles	Not naturally supported — under single inheritance a person is only ever one class at a time.	Naturally supported — User simply holds a set of Role objects, which directly matches the stated requirement that a person may hold more than one role.

Ease of changing permissions at runtime	Poor — changing a person's role effectively means replacing the object with a different subclass instance.	Good — a Role instance can be added, removed, or allowed to expire (validUntil) without touching the User object at all.
--	--	--

4.3 Chosen design and justification

This model uses Option B, composition with role objects. The scenario states as a fact that a person may hold more than one role at the same time and that roles and permissions may change during the semester. Inheritance cannot represent simultaneous multiple roles without violating the single-parent identity of an object, or resorting to multiple inheritance, which Java classes do not support. Role objects satisfy this requirement directly: a User's rights are simply the union of the permissions of every currently active Role it holds, and a mid-semester role change is just adding, removing, or expiring one Role instance rather than replacing the User.

Task 5 — Behaviour and Edge Cases

5.1 Reservation life cycle

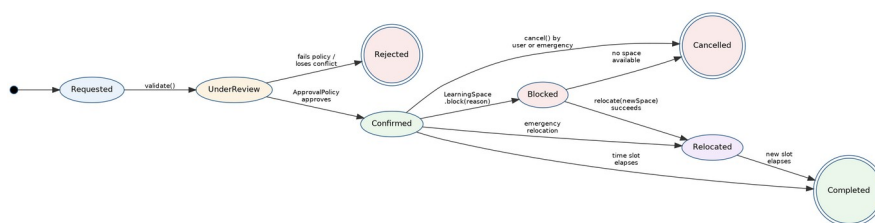


Figure 2 — State diagram for the life cycle of a Reservation

5.2 Handling the three required edge cases

Case 1 — Two simultaneous requests for the same space. `ApprovalPolicy.resolveConflict()` compares the two requests by priority tier first (Task 1, Conflict 1), then by the timestamp of validated submission within the same tier. The losing request is transitioned to `Rejected` and `NotificationService` informs that user; `AuditLog` records the decision and the reason.

Enforcing object: `ApprovalPolicy` — it is the single class that owns the conflict rule, so `User` and `Reservation` never need to know how conflicts are resolved.

Case 2 — A maintenance block after confirmation. Maintenance staff (via their active `Role`) call `LearningSpace.block(reason)`. `LearningSpace` itself transitions every `Confirmed` reservation tied to that space and overlapping time to `Blocked`, and `NotificationService` notifies each affected user; if no equivalent space can be found, the reservation moves to `Cancelled` instead.

Enforcing object: `LearningSpace` — it is the sole authority over its own physical/safety status, so the block cannot be second-guessed by `Reservation` or by an administrator trying to preserve utilization.

Case 3 — An emergency cancellation or relocation. An emergency event (for example, a building closure) is raised against one or more `LearningSpace` objects. The system walks every `Confirmed` reservation for the affected spaces and either relocates it to an equivalent-capacity space or cancels it if none exists. `AuditLog` stores the original assignment and the reason for the change, and `NotificationService` informs every affected user.

Enforcing object: `Reservation.relocate()/cancel()`, coordinated by `ApprovalPolicy`, with `AuditLog` and `NotificationService` — `Reservation` exposes only the guarded transition methods, so the emergency process can never silently overwrite booking state without leaving a record and a notification.

Task 6 — Independent Exploration

6.1 Role-Based Access Control (RBAC)

Role-Based Access Control is an approach to permissions in which rights are not attached directly to an individual person, but to a role, and a person gains those rights only by being assigned to one or more roles. Instead of asking “what can this specific user do,” the system asks “what can this role do,” and a user's effective permissions become the union of the permissions of every role currently assigned to them (Sandhu, Ferraiolo and Kuhn, 2000; NIST Computer Security Resource Center, n.d.).

This concept maps directly onto the Role class already used in Task 2. Rather than hard-coding a rule such as “if the user is Faculty, allow priority booking” inside Reservation or ApprovalPolicy, every permission check calls Role.hasPermission(action) against the caller's currently active roles. This keeps permission logic in a single place, lets one person hold several roles at once (for example, student and teaching assistant)

6.2 References for this section

- R. Sandhu, D. F. Ferraiolo and D. R. Kuhn, “The NIST Model for Role-Based Access Control: Towards a Unified Standard,” National Institute of Standards and Technology, 2000. Available: <https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>
- National Institute of Standards and Technology, “Role Based Access Control,” NIST Computer Security Resource Center project page. Available: <https://csrc.nist.gov/projects/role-based-access-control>

References

- Association for Computing Machinery and IEEE Computer Society, Computing Curricula 2020: Paradigms for Global Computing Education, 2020. Available: <https://www.acm.org/education/curricula-recommendations>
- Seoul Accord, Section D Graduate Attributes, especially Section D.4.1 Range of Problem Solving. Available: <https://www.seoulaccord.org/document.php?id=79>
- R. Sandhu, D. F. Ferraiolo and D. R. Kuhn, “The NIST Model for Role-Based Access Control: Towards a Unified Standard,” National Institute of Standards and Technology, 2000. Available: <https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard>
- National Institute of Standards and Technology, “Role Based Access Control,” NIST Computer Security Resource Center project page. Available: <https://csrc.nist.gov/projects/role-based-access-control>

Generative AI Use Disclosure

Claude (Anthropic), an AI assistant, was used while preparing this submission to help organise the requirement analysis, draft the UML class and state diagrams, and structure the written explanations across Tasks 1 to 6. The student specific condition (Privacy first, from a registration number ending in 0), every assumption, design decision, invariant, and the final choice of design in Task 4 were reviewed by the student, who remains fully responsible for the correctness, originality, and academic integrity of every claim made in this document.